



Dokumentace k projektu

Implementace interpretru imperativního jazyka SOL26

Předmět:

IPP

Jméno:

Patrik Lošťák

Login:

xlostap00

Rok:

2026

Varianta:

Python, TypeScript

Brno, 20. dubna 2026

Obsah

1	Úvod	2
2	Návrh a implementace	2
3	Hlavní interní datové struktury	3
4	Využití návrhových vzorů a principů OOP	3
4.1	Návrhový vzor Tovární metoda	3
4.2	Návrhový vzor Strategie	3
5	Největší problémy a jejich řešení	4
6	Úpravy a vylepšení	4
7	Rozšíření	4
7.1	Mechanismus doesNotUnderstand	4
7.2	Podpora rozhraní	4
8	Závěr	5
	Příloha: UML diagram tříd	5

1 Úvod

Cílem projektu bylo vytvořit program v programovacím jazyce Python (moje varianta), který provede interpretaci zdrojového jazyka. Zdrojový jazyk SOL26 je imperativní, nety-povaný, třídní objektově orientovaný programovací jazyk s dynamickou typovou kontrolou.

Samotný program funguje jako jednoduchá konzolová aplikace, která načte řídicí program v jazyce SOL26 ve formátu SOL-XML ze vstupu definovaného podle parametrů příkazové řádky, provede interpretaci a následně generuje jeho výsledek na standardní výstup; všechny ostatní výpisy, varování a chybová hlášení jsou posílána na standardní chybový výstup.

2 Návrh a implementace

Před samotnou interpretací se nejdříve pomocí metody `load_program()` otevře zdrojový soubor ve formátu SOL-XML, poté dojde k jeho validaci a na základě definovaných modelů v modulu `input_models.py` se z něj sestaví AST (abstraktní syntaktický strom). Před spuštěním programu pomocí statické sémantické kontroly implementované v metodě `_static_check()` probíhá ověření průchodem AST, zda existuje povinná třída `Main` a její instanční metoda `run`.

V další fázi je naplněna globální paměť pro tabulku tříd (`class_table`). Do ní jsou nejdříve zapsány všechny vestavěné třídy jazyka dle specifikace (např. `Integer`, `Block`...) a dále inicializovány sdílené instance (pro konstantní hodnoty `True`, `False`, `Nil`). Poté interpret projde všechny uživatelské třídy z AST a zkontroluje statické sémantické chyby (redefinice tříd, kolize selektorů metod, špatná arita, cyklická dědičnost...). Tyto třídy jsou následně uloženy do tabulky v podobě pomocných struktur `SolClass`.

Vstupním bodem provádění je metoda `execute()`, ve které se po všech předchozích krocích jako první vytvoří instance (objekt třídy `Main`) a dále se založí hlavní lokální paměťový rámec (`LocalFrame`) s proměnnou `self`, který zajišťuje správnou správu kontextu proměnných, předávání parametrů a vytváření zásobníku volání. Každý rámec si uchovává odkaz na své předky (`parent_frame`), čímž se podařilo vyřešit podporu uzávěrů u zanořených bloků. Poté se už začne vyhodnocovat příkaz po příkazu pro metodu `run`. V hlavním cyklu se podle typu zpracovávaného AST uzlu rekurzivně volají speciální metody:

- `eval_assign()`: Vyhodnocuje výrazy na pravé straně a ukládá výsledek do proměnné. Kontroluje, aby se nezapisovalo do klíčových slov `self` nebo `super`, a dále hledá, do jakého zanořeného rámce má hodnotu uložit.
- `eval_expr()/_var_expr()`: Vyhodnocuje jednotlivé výrazy. Překládá XML literály na skutečné datové typy z Pythonu. Umí brát hodnotu z lokální proměnné (u uzávěru i z vnějšího bloku) nebo přistoupit přímo k instančnímu atributu objektu.
- `eval_send()`: Viz níže.

Samotným jádrem interpretru je posílání zpráv, o které se stará metoda `eval_send()`. Když se pošle zpráva objektu, metoda prohledá posloupnost tříd od příjemce zprávy směrem k jeho předkům. Pokud se jedná o zprávu uživatelskému objektu, vytvoří se nový paměťový rámec a postupně se vyhodnotí všechny příkazy bloku dané metody. Když se jedná o vestavěný typ, o zpracování se postará metoda `_eval_builtin_send()`. Pro operace vyžadující instanci objektů (např. zpráva `new`) se vyhodnocení předá metodě `_eval_cls_msg()`. V případě, že `eval_send()` nenašlo žádnou metodu, pokusí se příkaz brát jako přístup ke čtení (getter) nebo zápisu (setter) instančního atributu objektu.

3 Hlavní interní datové struktury

- **SolClass**: Reprezentuje definici třídy v paměti interpretru. Drží v sobě název, odkaz na předka a referenci na původní AST uzel třídy z šablony.
- **SolInst**: Reprezentuje danou instanci objektu v paměti. Obsahuje odkaz na svůj mateřský **SolClass**, slovník pro instanční atributy (`attrs`) a případně i speciální položku `val` pro uložení surových dat.
- **LocalFrame**: Implementuje kontext proměnných pro metody a bloky a zásobník volání. Každý rámec uchovává svůj slovník lokálních proměnných, kolekci názvů parametrů (k detekci "shadowing") a odkaz na nadřazený rámec (`parent_frame`), což řeší funkčnost uzávěrů.

4 Využití návrhových vzorů a principů OOP

4.1 Návrhový vzor Tovární metoda

Tento vzor byl využit pro zapouzdření a zjednodušení logiky vytváření nových paměťových instancí objektů. Jako tvůrce zde vystupuje hlavní třída interpretru a jako konkrétní tovární metody pak metody jako `_cls_msg_new()`, `_cls_msg_from()`, `_cls_msg_read()`, které vracejí univerzální produkt v podobě třídy **SolInst**. Tovární metoda dokáže na základě přijaté zprávy a cílové třídy rozhodnout o správné inicializaci. Tento přístup usnadňuje budoucí rozšiřitelnost – pokud by přibyl nový vestavěný typ, většina změn by se tak omezila na tělo jediné tovární metody.

4.2 Návrhový vzor Strategie

Pro dynamické vyhodnocování vestavěných matematických a logických operací u třídy **Integer** byly použity principy návrhového vzoru Strategie. Roli kontextu plní metoda `_builtin_integer()`, která jako strategie využívá vestavěné funkce ze standardního modulu `operator` (např. `operator.add`, `operator.sub`). Tyto algoritmy jsou napojeny na konkrétní selektory zpráv pomocí slovníku `math_ops`. Tento vzor úplně eliminoval dlouhý

rozhodovací blok (`if-elif-else`) a pro rozšíření o další operátor stačí pouze přidat nový záznam do slovníku.

5 Největší problémy a jejich řešení

Největším problémem byla implementace řešení bloků s udržení správného lexikálního rozsahu proměnných (tzv. uzávěry). V prvotní verzi program používal pouze jeden paměťový rámec, který však nefungoval správně v situaci, kdy se v zanořeném bloku mělo přistupovat k proměnným definovaným ve vnější metodě. Tento problém vyřešila úprava návrhu třídy `LocalFrame`, kde nyní každý rámec může obsahovat odkaz na svého přímého předka (`parent_frame`). V průběhu vyhodnocování příkazů, vytváření proměnných nebo čtení jejich hodnot se interpret pokusí, v případě nenalezení proměnné v aktuálním rámci, rekurzivně prohledat předky.

6 Úpravy a vylepšení

Na základě výsledků referenčních testů, kdy u testeru neprošla většina testů kvůli zapisování do chráněného souborového systému (chyba `EROFS`), proběhla úprava logiky generování pomocných souborů. Nyní se vše generuje do systémové dočasné složky pomocí metody `os.tmpdir()`. V interpretaci došlo k upřesnění a rozšíření statické sémantické analýzy, konkrétně k úpravě kontroly duplicitních selektorů metod v rámci jedné třídy, byla přidána kontrola cyklické dědičnosti a opraven chybový kód u redefinice tříd.

7 Rozšíření

7.1 Mechanismus `doesNotUnderstand`

K implementaci tohoto rozšíření by stačilo upravit metodu `eval_send()`, která se stará o celkové dynamické volání metod. Momentální implementace při nenalezení metody vyhazuje výjimku s kódem 51. Pro podporu tohoto rozšíření by stačilo chybovou větev nahradit tak, že by se vytvořila nová instance (`SolInst`) vestavěné třídy `Message`, do které by se zapouzdřily argumenty a původní selektor. Poté by `eval_send()` rekurzivně volal sám sebe s novým selektorem `doesNotUnderstand`: a předal zprávu jako argument. Stávající ošetření chyby 51 by se přidalo do metody `_builtin_object()`. Tento návrh eliminuje nutnost zásahu do uzlů AST a do správy paměťových rámců.

7.2 Podpora rozhraní

Pro podporu rozhraní by se musela přidat reprezentace do struktury AST, dále by se fáze kontroly vložila za fázi naplnění tabulky tříd (`class_table`). Díky tomu, že struktura `SolClass` již uchovává odkaz na uzel AST s definicemi metod a tabulka tříd obsahuje kompletní mapu dědičnosti, stačilo by před spuštěním hlavní metody přidat kontrolu.

Tato kontrola by pro každou třídu procházela přes daná rozhraní a pomocí logiky, (jako v `eval_send()`) by v posloupnosti dědičnosti staticky ověřovala přítomnost vyžadovaných selektorů. Zákaz instanciací by se pak vyřešil přidáním jedné podmínky do metody `_cls_msg_new()`.

8 Závěr

Projekt jako celek mi pomohl proniknout do světa objektově orientovaného programování a návrhových vzorů, seznámit se se základními principy a také objevit některá úskalí a problémy, které jsem musel během implementace řešit. Výhodou bych pak bral seznámení se s úplně novým jazykem (TypeScript) při implementaci testovacího nástroje. Během vývoje jsem využíval LLM jako interaktivního asistenta. Nástroj nebyl použit k generování ucelených bloků řešení, ale sloužil hlavně pro konzultace logických řešení, návrhu architektury a debugging.

Příloha: UML diagram tříd

